Documentation Duality and AI Collaboration

Documentation Duality and AI Collaboration I

The Documentation Duality standard pairs two files in documented source directories; the live inventory reports actual coverage:

Documentation Duality and AI Collaboration I

- `README.md`: Human-readable overview, quick-start instructions, and directory structure.
- `AGENTS.md`: Machine-readable technical specification optimized for AI coding assistants. Contains API tables, dependency graphs, implementation patterns, and architectural constraints.

This Documentation Duality standard serves two purposes. First, it ensures that both human researchers and AI agents can navigate the codebase efficiently—`AGENTS.md` files provide the structured context that language models need to make informed code modifications without hallucinating API signatures or violating architectural invariants. Second, it creates a self-documenting feedback loop: as AI agents modify the codebase, they update the corresponding `AGENTS.md` files, keeping documentation synchronized with implementation. Lau and Guo's survey of 90 AI coding assistant systems (Lau and Guo 2025) identifies contextual code understanding as a primary bottleneck; the Documentation Duality standard addresses this by providing pre-structured context at every directory level.

Root `AGENTS.md` defines shared architectural and testing constraints. `.agents/` supplies generic skills, roles, workflows, and confined local state; `.agents/AGENTS.md` documents that catalog. Per-directory `AGENTS.md` adds local contracts, and `README.md` supplies human navigation. `CLAUDE.md` is a client command reference.

Agentic Skill Architecture

Agentic Skill Architecture I

The Documentation Duality standard addresses human and AI navigation at the directory level. A complementary layer operates at the *module* level: capability-specific `SKILL.md` descriptors add discoverable instructions and invocation examples to documented modules. A descriptor supplies context; executable tools remain separately registered interfaces.

The Three-Tier Skill Protocol I

`template/` implements a progression of agent-facing documentation, escalating in specificity from global constraints to module-level API contracts:

The Three-Tier Skill Protocol I

Tier	File	Scope	Purpose
1 — System	Root `AGENTS.md`, `.agents/`	Repository-wide	Shared constraints, skills, roles, workflows, and local state
2 — Structure	`AGENTS.md`	Documented directories	Local file inventories, API surfaces, integration patterns, architectural constraints
3 — Skill	`SKILL.md`	Skill-enabled modules and exemplars	Machine-parseable skill descriptor: module name, description, key imports, usage pattern

Tier 1 and Tier 2 have direct analogues in the prompt-engineering literature: system prompts and retrieval-augmented context (Lau and Guo 2025). Tier 3 is novel. The `SKILL.md` files combine YAML discovery metadata with Markdown instructions. These descriptors complement the separately registered tools exposed through the Model Context Protocol (Anthropic 2024). The following presents the metadata from `infrastructure/rendering/SKILL.md`, with its description folded into equivalent YAML for readability:

The Three-Tier Skill Protocol I

```
---
name: infrastructure-rendering
description: >-
  Skill for the rendering infrastructure module
providing multi-format
  output generation including PDF manuscripts, HTML
web pages, and
  Beamer/Reveal.js slides. Use when rendering
research outputs,
  converting markdown to PDF, generating slides, or
configuring LaTeX
  rendering.
---
```

A skill loader reading this block discovers the module name and its natural-language activation condition. The Markdown body supplies Python imports and usage examples; the frontmatter does not itself register an executable MCP tool. This is the practical implementation of Toolformer-style self-documented tools (Schick et al. 2023)—rather than a language model learning tool APIs from training data, the APIs are encoded directly in version-controlled, co-located skill files that evolve with the codebase.

Module Skill Coverage I

`discover_infrastructure_modules()` reports the actual `README.md`, `AGENTS.md`, `SKILL.md`, and `PAI.md` presence for each enumerated subdirectory. `python -m infrastructure.skills` maintains the shared `.agents/skill_manifest.json`; module and public-exemplar skills participate independently of any editor. `PAI.md` provides an optional integration narrative.

Promotion policy: new Layer-1 directories must ship human + machine-readable docs (`README.md`, `AGENTS.md`) immediately; Tier-3 SKILL assets follow once APIs stabilize.

MCP Server Mapping I

The opt-in `infrastructure.mcp_server` exposes the existing operation catalog, pipeline description, discovered skills, and curated CLI invocation through `list_operations`, `describe_pipeline`, `list_skills`, and `invoke_cli`. CLI invocation is confined to registered infrastructure modules; mutating or external operations require the server's explicit opt-in. Skill discovery and tool execution therefore remain separate contracts. The server does not automatically expose every Python function, rendered file, or suggested module-to-resource mapping.

An agent can inspect these catalogs and choose supported operations while retaining the pipeline's declared dependencies and acceptance gates. Discovery alone does not establish successful execution or publication. The `SKILL.md` files parallel Voyager's skill library (Wang et al. 2023)—Voyager's agent accumulates a growing library of executable Minecraft skills represented as JavaScript functions; `template/`'s agent accumulates a curated library of research pipeline skills represented as YAML-frontmattered `SKILL.md` files. In both cases, the skill representation is machine-readable, version-controlled, and self-describing. Wang et al.'s LLM agent survey (Wang et al. 2024) identifies tool use, planning, and memory as the three fundamental capabilities of autonomous agents; Yao et al.'s ReAct framework (Yao et al. 2023) demonstrates that interleaving reasoning traces with tool actions dramatically improves

agent reliability in interactive settings. The `template/` skill architecture maps cleanly onto these three capabilities: the `SKILL.md` descriptors supply the tool-use layer, the declarative DAG of 17 `pipeline.yaml` stages (a default full run executes 10) supplies the planning scaffold, and the per-criterion checkpoint system supplies the memory layer.